

Project Design Phase

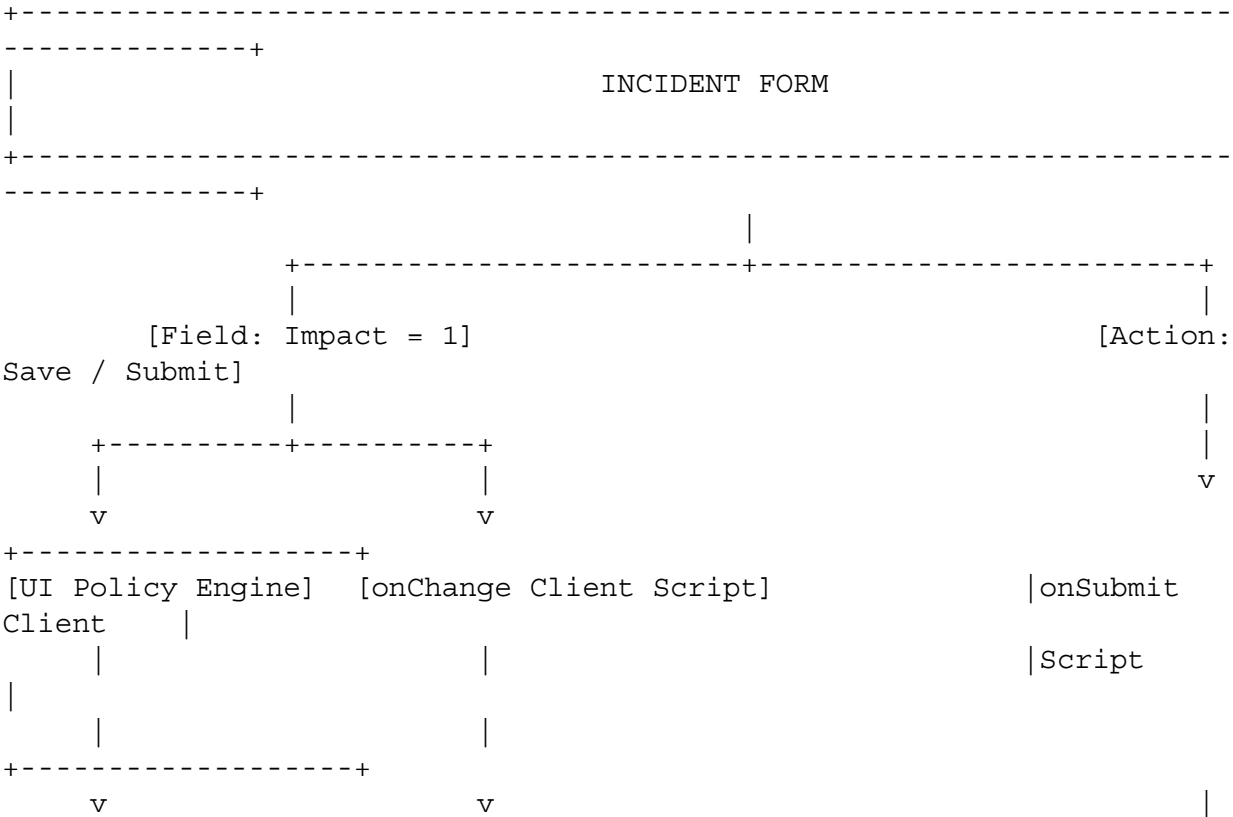
This phase outlines the architectural layout, component relationships, execution flow, and design patterns utilized in the **Implement Client Script & UI Policy (Incident)** implementation.

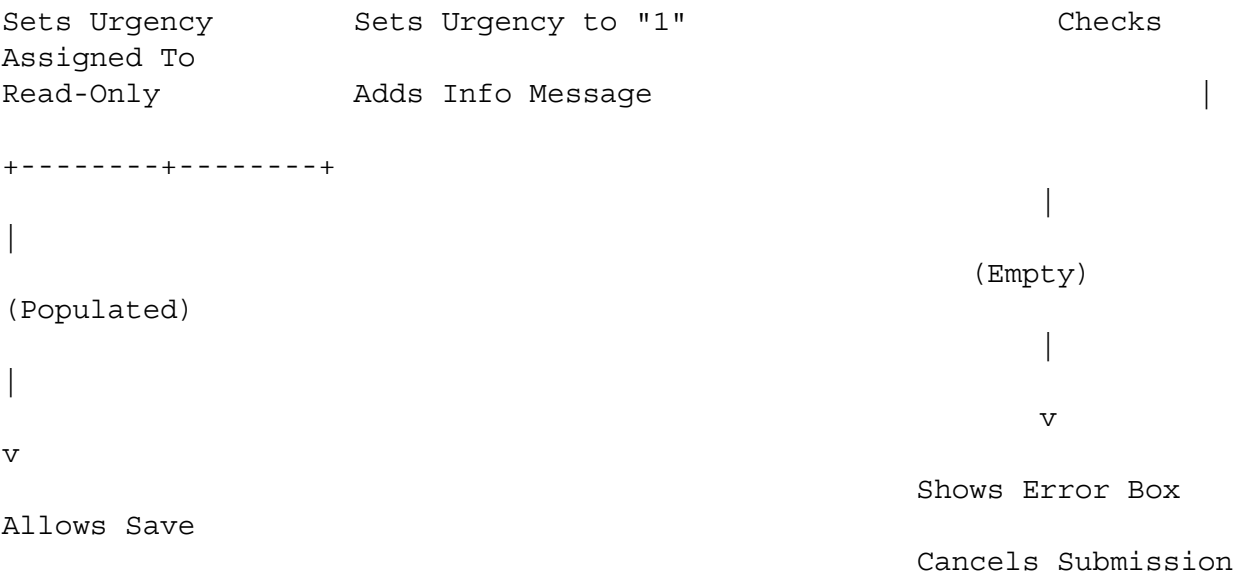
1. Component Architecture & Design Patterns

The implementation follows a standard client-side architecture in ServiceNow, using two primary mechanisms: declarative UI Policies for field properties and imperative Client Scripts for complex logic and events.

- **UI Policy Layer (Declarative Engine):**
 - Handles field state properties (Read-only, Mandatory, Visible) dynamically without writing code.
 - Configured with **Reverse if false** to ensure automatic rollback when conditions are no longer met.
- **Client Scripting Layer (Imperative Engine):**
 - Executes event-driven JavaScript directly in the user's browser via the g_form API.
 - Handled via three standard event types:
 - onChange: Triggered upon user modifying the Impact field.
 - onSubmit: Intercepts the form submission process to validate inputs.
 - onCellEdit: Intercepts modifications performed directly on list views.

2. Component Specification & Workflow Matrix





Component Details

Component Name	Type	Event / Trigger	Design Purpose	Execution Scope
High Impact Control	UI Policy	Impact is 1 - High	Enforces read-only state on Urgency dynamically.	Incident Form (Client-Side)
Auto set urgency for high impact	Client Script	onChange (impact field)	Programmatically sets urgency to 1 and notifies user.	Incident Form (Client-Side)
Prevent save if Assigned To missing	Client Script	onSubmit	Validates assigned_to when impact == '1' and halts submit if missing.	Incident Form Submission
Prevent state change via list edit	Client Script	onCellEdit (state field)	Blocks list view edits to state via alert and callback(false).	Incident List View

3. Detailed Logic Sequence

A. Form Interaction Flow (onChange & UI Policy)

- 1. User changes Impact to 1 (High).
- 2. **UI Policy Engine** detects condition match (Impact == 1):
 - Sets Urgency field to **Read-Only**.
- 3. **onChange Script** executes:
 - Evaluates if (isLoading || newValue == "") -> Exit if true.
 - Evaluates if (newValue == '1'):
 - Executes g_form.setValue('urgency', '1').

- Executes `g_form.addInfoMessage(...)` to display confirmation.

B. Record Submission Flow (onSubmit)

1. User clicks **Submit** or **Update** on the Incident form.
2. **onSubmit Script** executes:
 - Evaluates condition: `g_form.getValue('impact') == '1' && g_form.getValue('assigned_to') == ''`.
 - **If True:**
 - Triggers `g_form.showErrorMessage('assigned_to', 'Assigned To is mandatory for High impact incidents.')`.
 - Returns false to abort the save sequence.
 - **If False:**
 - Returns true; submission proceeds.

C. List Editing Flow (onCellEdit)

1. User attempts to double-click and edit the State column in an Incident list.
2. **onCellEdit Script** executes:
 - Displays browser alert: 'State cannot be updated using list editing. Please open the Incident.'
 - Invokes `callback(false)` to discard changes and cancel list update.